

## Lab 3 Report

112062119 楊竺函

### Proj04-01: Two-Dimensional Fast Fourier Transform

#### 1. Implementation

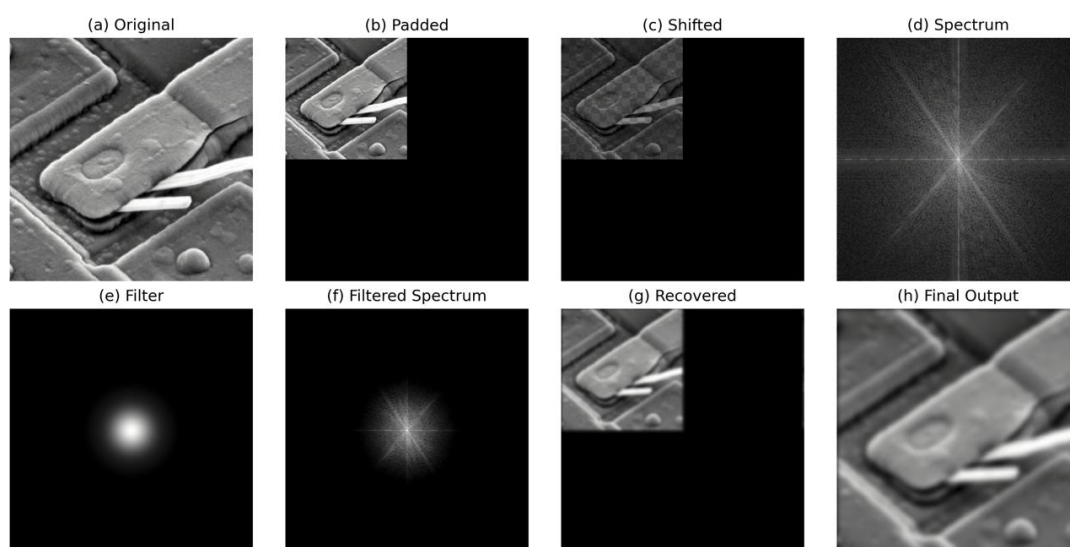
##### 1.1 主程式實作

- (1) 程式先詢問使用者是否需要 zero padding，作為後續流程分支的依據。
- (2) 使用 `cv2.imread()` 讀入灰階影像後，將影像轉為 `float32` 並正規化到 `[0, 1]`，再利用 `cv2.resize()` 將原圖縮小為  $256 \times 256$ 。
- (3) 若使用 zero padding，則呼叫 `zeroPad()` 將影像擴展為原本兩倍大小；接著呼叫 `display_adjustment()` 將 padded 結果轉為可顯示影像並輸出成檔案。
- (4) 若有 padding，則對 padded 後的影像呼叫 `shiftSpectrum()`；否則直接對原影像呼叫 `shiftSpectrum()`，將低頻成分移至頻譜中心。之後把結果轉成可儲存的影像格式輸出。
- (5) 呼叫 `DFT.myDFT2()` 對中心化後的影像做二維離散傅立葉轉換，得到頻域表示。再呼叫 `display_adjustment(..., True)` 對複數頻譜做 magnitude、log transform 與 normalization，輸出頻譜圖。
- (6) 呼叫 `LowPass.myGLPF(30, *shifted_img.shape)` 產生與輸入影像同尺寸的 Gaussian lowpass filter，再透過 `display_adjustment()` 將 filter 正規化後輸出成圖。
- (7) 將頻譜 `dft_img` 與濾波器 `H` 逐點相乘，得到濾波後的頻譜 `filtered_dft`。之後同樣呼叫 `display_adjustment(..., True)` 將其轉成可視化頻譜圖並存檔。
- (8) 呼叫 `DFT.myIDFT2()` 將濾波後頻譜轉回空間域，再呼叫 `DFT.shiftSpectrum()` 乘回  $(-1)^{(x+y)}$ ，將影像恢復到正常排列位置。
- (9) 若有做 padding，會先呼叫 `display_adjustment()` 將尚未裁切的 recovered image 輸出成 `filtered_raw.tif`，接著取左上角  $256 \times 256$  區域作為最終結果，再輸出 `output_zero_pad.tif`。若未使用 padding，則直接呼叫 `display_adjustment()` 將 recovered image 轉為顯示格式並輸出成 `output.tif`。
- (10) 若有使用 zero padding，則利用 `show_img()` 將原圖與上述流程中產生的輸出影像排成一張  $2 \times 4$  的圖，最後存成 `result.png`。

## 1.2 副程式實作:

- (1) **zeropad(input)**: 將給定的影像 `input` 放大為原本的兩倍，用 0 填充除了原始影像所在的位置以外的所有像素。
- (2) **shiftSpectrum(input)**: 對輸入影像 `input` 逐點乘上  $(-1)^{(x+y)}$ 。
- (3) **dft1d(x)**: 建立 DFT matrix  $W$ ，並以矩陣乘法  $W@x$  計算一維 Fourier Transform。
- (4) **myDFT2(input)**: 採用 two-pass 方法：先對每一列做 1D DFT，再對每一行做 1D DFT，完成 2D DFT 計算。
- (5) **myIDFT2(input)**: 利用共軛性質先對輸入取共軛並做 DFT，再取共軛並除以  $MN$ ，實現 2D 反傅立葉轉換，最後取實部輸出。
- (6) **show\_img(ax, img, title)**: 使用 matplotlib 在指定的 subplot (`ax`) 上顯示灰階影像，並設定標題與關閉座標軸以提升視覺呈現。
- (7) **display\_adjustment(img, iscomplex=False)**: 若為複數頻譜，先取 magnitude 並進行 log transform 壓縮動態範圍，再做 normalization 映射至  $[0, 255]$ ；若為一般影像，則直接做 min-max normalization 轉換為可顯示的 8-bit 灰階影像。
- 註: 程式所用的 lowpass filter 為在 Proj04-03 中所實作的 myGLPF()

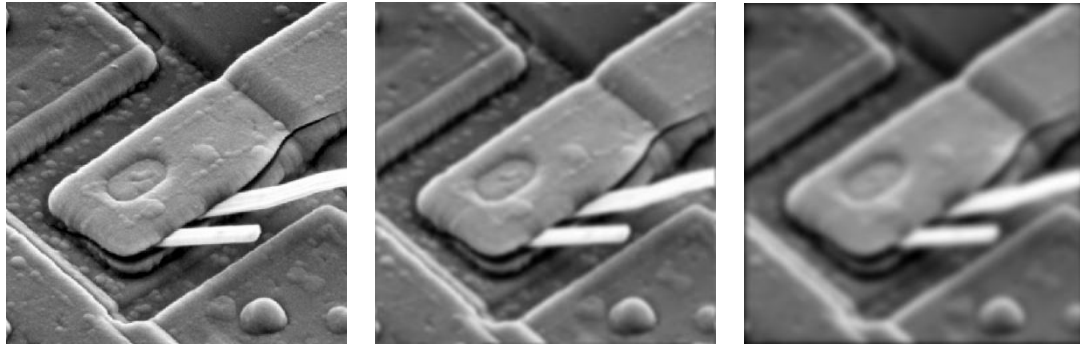
## 2. Execution Result



註: (e) 圖所用的 cut-off frequency  $D_0 = 30$

### 3. Discussion

#### 3.1 分析比較轉到頻率域前有 padding 和沒有 padding 的結果圖片



original

without zero padding

with zero padding

我採用固定 Gaussian lowpass filter 的 cutoff frequency  $D_0$ ，比較在進行頻率域轉換前有無 zero padding 對最終影像的影響。實驗結果顯示，未使用 padding 的情況下，輸出影像相對較清晰，保留較多細節；而使用 padding 時，影像整體較為平滑，呈現較強的模糊效果，但邊界過渡較自然。其原因在於，zero padding 會將影像尺寸由  $256 \times 256$  擴展至  $512 \times 512$ ，在相同的  $D_0$  下，filter 在頻率域中的相對範圍變窄，使得更多高頻成分被抑制，導致影像更模糊。此外，padding 可降低 DFT 所造成的週期性延拓影響，使 frequency domain filtering 更接近 linear convolution，因此邊界 artifact 較少。總結而言，zero padding 改善了邊界行為，但在固定  $D_0$  下會使濾波效果相對變強，造成視覺上較模糊的結果。

#### 3.2 討論能加速 DFT 計算的方法

在本次 lab 中，我在實作 2-D DFT 時，並未直接使用最原始的定義式進行計算，而是採用了 two-pass 1-D DFT 搭配矩陣運算的方式，以提升計算效率。

若直接根據 2-D DFT 的定義式：

$$F(u, v) = \sum_x \sum_y f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

以四層巢狀 for loop 實作，時間複雜度約為  $O(N^4)$ 。對於影像大小為  $256 \times 256$  的情況，運算量極大，實際執行時間會非常緩慢，幾乎不具實用性。

因此，我採用 two-pass 1-D DFT 的方式，利用 DFT 的可分離性，先對每一列進行 1-D DFT，再對每一行進行 1-D DFT，將原本的 2-D DFT 分解為兩次 1-

D DFT，使時間複雜度降低為  $O(N^3)$ 。相較於直接使用四層迴圈可以顯著減少運算時間。此外，在 1-D DFT 的實作中，我使用了矩陣乘法取代內層的 for loop。透過預先建立 DFT matrix，並使用向量化運算，將原本的逐項累加轉為矩陣運算，使計算能夠由 NumPy 底層最佳化函式處理。雖然理論上的時間複雜度仍為  $O(N^2)$ ，但由於避免了 Python 層級的迴圈，實際執行速度可大幅提升。

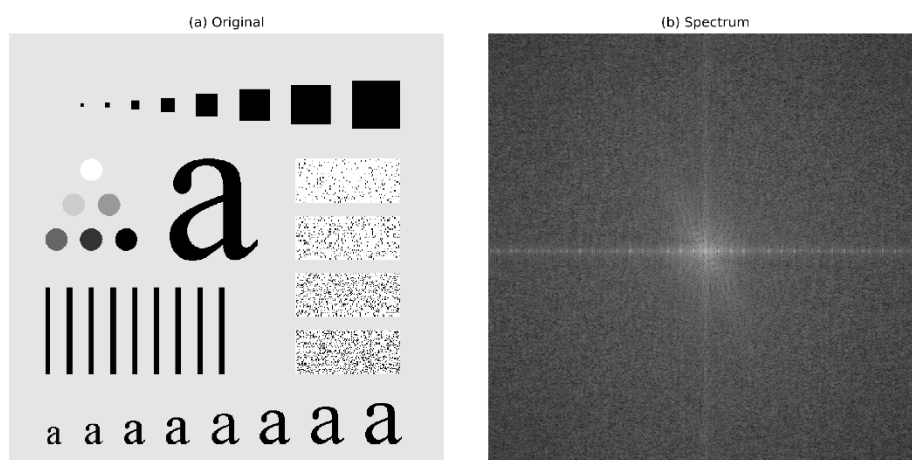
## **Proj04-02: Fourier Spectrum and Average Value**

### **1. Implementation**

- (1) 使用 `cv2.imread()` 讀入影像，轉為 `float32` 並正規化到 `[0, 1]`。
- (2) 呼叫 `shiftSpectrum()` 將影像乘上  $(-1)^{(x+y)}$ ，把低頻移到頻譜中心。
- (3) 使用 `np.fft.fft2()` 對中心化後影像做傅立葉轉換，並呼叫 `display_adjustment()` 將頻譜轉為可顯示影像後存檔。
- (4) 直接使用 `np.mean()` 計算原影像的平均灰階值。
- (5) 利用頻譜中心點  $F(M/2, N/2)$ ，除以  $MN$  取得影像平均值。
- (6) 將直接計算與頻域計算的平均值印出比較。
- (7) 呼叫 `show_img()` 將原圖與頻譜圖並排顯示，並存成 `result.png`。

註: `shiftspectrum()`, `display_adjustment()` 與 `show_img()` 皆沿用上題實作內容，故此處不贅述。

### **2. Execution Result**

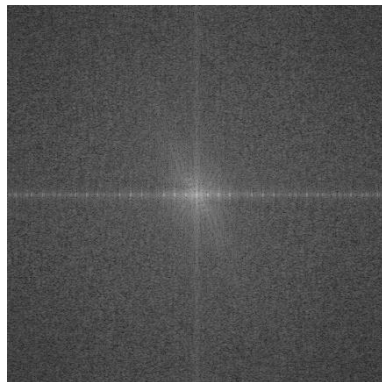


### 3. Discussion

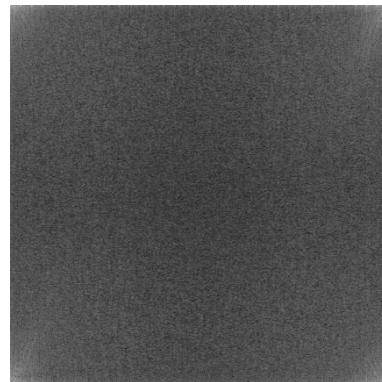
#### 3.1 比較直接從圖片算 mean 跟由 spectrum 中心點資訊得到 mean 的不同

直接由影像像素計算平均值，是在空間域中將所有像素值加總後除以影像大小 ( $M \times N$ )；而由 Fourier spectrum 中心點 (DC 成分) 計算平均值，則是利用頻域中  $F(0,0)$  代表整張影像像素總和的特性，再除以 ( $M \times N$ ) 得到平均值。兩種方法在理論上是完全等價的，因此計算結果應相同；但在實作上可能出現極小差異，主要是來自 FFT 過程中的浮點數精度誤差。

#### 3.2 Effect of Centered Operation



centered spectrum



uncentered spectrum

我稍微修改了一下的程式讓輸入的影像不要做 centered 處理，以此來比較未經 centered 處理 (uncentered spectrum) 與經過 centered 處理後的 Fourier spectrum 在顯示上的差異。當未進行 centered 時，頻譜的 DC 成分會位於左上角位置  $(0,0)$ ，導致低頻資訊集中在角落，使整體頻譜分布較不直觀；相較之下，經由將影像乘上  $(-1)^{x+y}$  進行 centered 處理後，頻譜會產生平移，使 DC 成分移至頻譜中央，低頻位於中心、高頻分布於外圍，這樣的排列方式更符合人類對頻率分布的直觀理解，也更容易觀察影像的頻率特性。

此外，centered 與 uncentered 並不改變頻譜本身的數值內容，而僅是位置上的重排，因此在計算影像平均值時需注意 DC 成分的位置。在未 centered 的情況下，DC 成分位於  $F(0,0)$ ，因此平均值可由  $F(0,0)/(M \times N)$  計算得到；而在 centered spectrum 中，DC 成分會移至頻譜中心位置  $(M/2, N/2)$ ，此時需改由該位置的值除以  $(M \times N)$  才能得到正確的平均值。此結果進一步說明 centered 操作本質上為頻譜的平移，不影響其所代表的物理意義。

## Proj04-03: Lowpass Filtering

### 1. Implementation

#### 1.1 主程式實作

(1) 先用 `plt.subplots(2, 3)` 建立  $2 \times 3$  的顯示版面，準備放置原圖與不同 cutoff frequency 下的濾波結果。

(2) 呼叫 `init()` 讀入影像、完成 padding、中心化與 DFT，並取得原始影像 `img` 與其頻譜 `dft_img`。

(3) 先將原始影像放入 `result_set`，作為之後顯示比較的第一張圖。

(4) 逐一走訪 `D0_list` 中的 cutoff frequency，對每個 `D0` 呼叫 `performLowPassFiltering(D0, img, dft_img)` 完成濾波，並將結果加入 `result_set`。

(5) 依序讀取 `result_set` 內的影像，依序標示為對應的標題，並呼叫 `show_img()` 顯示在對應位置。最後使用 `plt.tight_layout()` 調整版面後，輸出成 `result.png`。

#### 1.2 副程式實作

(1) **`myGLPF(D0, M, N)`**: 先建立大小為  $M \times N$  的座標矩陣，並計算每一點到頻譜中心  $(M/2, N/2)$  的平方距離；再代入 Gaussian lowpass filter 公式，產生對應的濾波器值。

(2) **`init()`**: 先讀入影像並轉成 `float32` 後正規化到  $[0, 1]$ ，再呼叫 `mirrorPad()` 做 mirror padding，並呼叫 `shiftSpectrum()` 將影像中心化。之後使用 `np.fft.fft2()` 對前處理後影像做 2D DFT，最後回傳原圖與其頻譜結果。

(3) **`mirrorPad(img)`**: 利用 `cv2.copyMakeBorder()` 將影像在上下與左右各擴展原尺寸大小，並採用 mirror（反射）方式填補邊界，使邊界像素呈現連續延伸。

(4) **`performLowPassFiltering(D0, img, dft_img)`**: 先根據輸入頻譜大小呼叫 `LowPass.myGLPF()` 產生 Gaussian lowpass filter，並將其與頻譜逐點相乘完成頻率域濾波。接著利用 `np.fft.ifft2()` 做反轉換，再呼叫 `shiftSpectrum()` 將影像移回正常位置，最後裁切回原始大小並呼叫 `display_adjustment()` 轉成可顯示影像後輸出並回傳可顯示影像。

註: `shiftspectrum()`, `display_adjustment()` 與 `show_img()` 皆沿用上題實作內容，故此處不贅述。

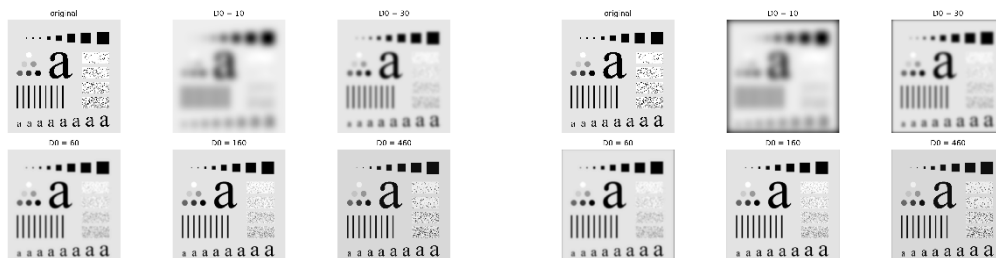
## 2. Execution Result



從執行結果可以看到隨著 cutoff frequency 提高，lowpass filtering 的效果越弱，影像也更接近原圖，符合公式 4-116 所預期的結果。

## 3. Discussion

### Different Padding Approaches



result using mirror padding

result using zero padding

在這題實作中我遵照了課本圖片文字敘述採用了 mirror padding 將影像擴展至較大的尺寸再進行頻域濾波。除此之外我也額外實驗了如果用 zero padding 會如何影響最終結果的呈現。Zero padding 會在影像邊界補上 0，使原本的灰階值在邊界處產生明顯的不連續變化，這種突變在頻域中會對應到額外的高頻成分，因此在經過濾波後，容易在影像邊緣產生黑邊或不自然的過渡現象。相對地，mirror padding 透過將影像在邊界進行對稱延伸，使影像在邊界處保持連續性，能有效減少因不連續所引入的高頻成分，因此濾波後的結果在邊界處會更加平滑且自然。兩者的對比在 cutoff frequency 較小時尤為明顯，推測原因可能是當 D0 較小時，濾波器僅保留低頻成分，使影像整體變得較為平滑，此時由 zero padding 所引入的邊界不連續會被低通濾波強調為明顯的亮暗差異，因而更容易觀察到邊

界的異常；而 mirror padding 因本身維持了邊界的連續性，即使在強低通濾波下仍能保持較自然的影像過渡。

綜合而言，padding 方法會直接影響頻域中所包含的頻率成分，進而影響濾波結果的視覺品質。相較於 zero padding，mirror padding 能有效降低邊界所引入的非必要高頻，得到更接近理想的濾波結果，因此在實務上通常是較佳的選擇。

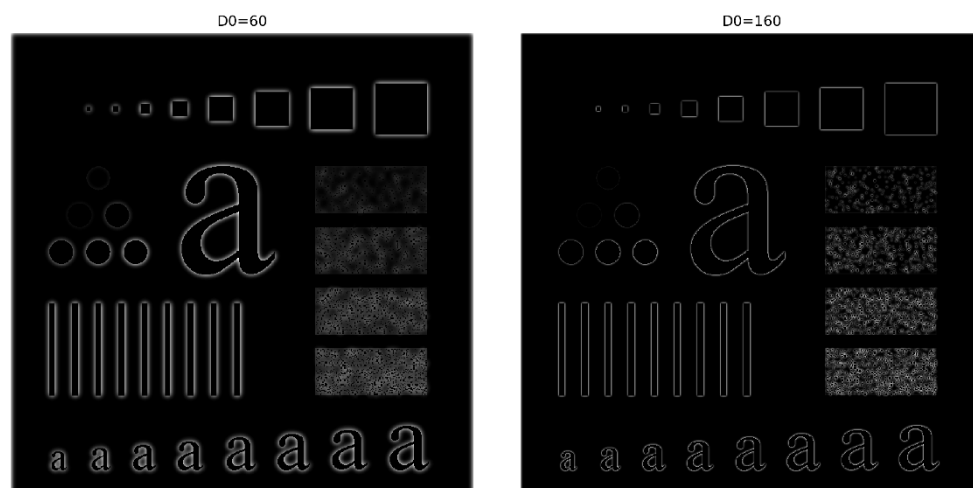
## **Proj04-04: Highpass Filtering**

### **1. Implementation**

本題的整體流程（包含影像讀入、padding、頻譜中心化、DFT、頻域濾波、IDFT、去中心化與裁切等步驟）皆與 Proj04-03 的 Implementation part 相同，因此此處不再贅述，僅針對以下不同之實作進行說明：

- (1) 本題改為實作 Gaussian Highpass Filter，公式為  $1 - (\text{原本的 lowpass filter})$ 。
- (2) Padding 方法不同：Proj04-03 採用 mirror padding，而本題則使用 zero padding。
- (3) 輸出影像處理方式不同：本題未使用 min-max normalization，而是直接透過 np.clip() 將結果限制在  $[0, 255]$ ，再轉為 uint8 輸出，以避免高通濾波造成的數值溢出影響顯示結果。

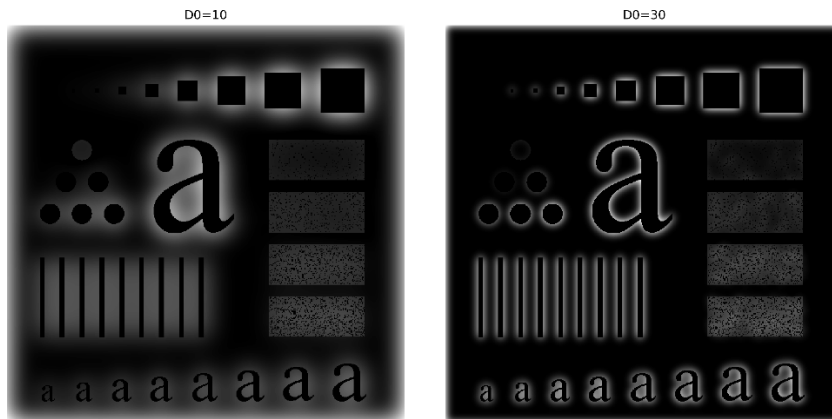
### **2. Execution Result**



從執行結果可以看到隨著 cutoff frequency 提高，highpass filtering 的效果越弱，影像也更接近原圖，符合公式 4-120 所預期的結果。



### 3. Discussion



在 highpass filtering 的實驗中，當 cutoff frequency ( $D0$ ) 設定為較小值（如  $D0 = 10$ ）時，觀察到影像邊緣周圍出現明顯的亮暈（halo）與模糊外擴現象。此現象主要是因為 highpass filter 在  $D0$  較小時會抑制幾乎所有低頻成分，使影像中原本負責平滑過渡與整體結構的資訊被大幅移除，只保留極高頻成分（如邊緣與細節）。在這種情況下，原本平滑的邊緣會轉變為具有震盪特性的訊號，導致空間域中出現過衝與振盪，進而形成邊緣周圍的亮暈效果。

此外，這種現象亦可從頻域與空間域轉換的角度理解：當在頻域中進行強烈的濾波（特別是大幅抑制低頻）時，會在空間域中產生類似 Gibbs phenomenon 的震盪現象，使得邊界不再平滑，而呈現不自然的強化效果。相較之下，當  $D0$  增加（如  $D0 = 30$  或更大）時，濾波器允許通過的頻率範圍變廣，部分低頻成分得以保留，使邊緣過渡較為平滑，影像整體也更加接近原始影像。因此，過小的 cutoff frequency 雖能強調邊緣，但同時也會引入失真與不自然的視覺效果，需在實際應用中適當選擇參數。